WEEK 4 SQL PROJECT

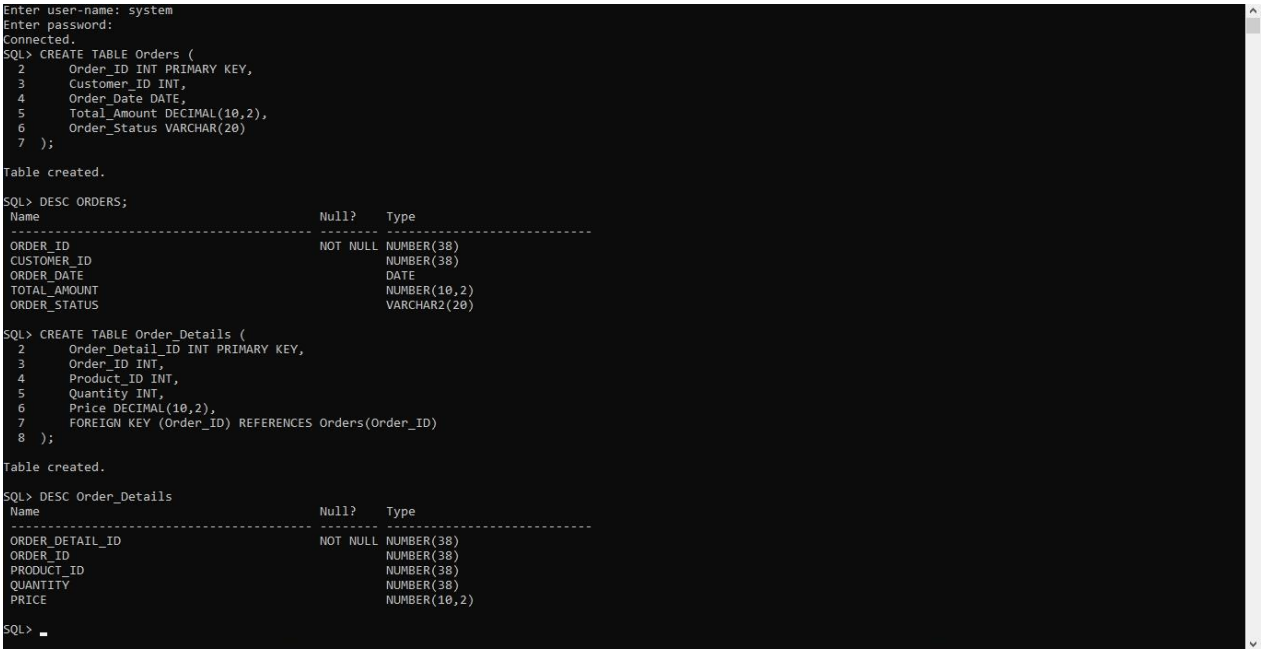
Order Management and Customer Purchase Analysis System

Scenario

The project implements an e-commerce order management database using Oracle SQL. The system stores customer information, product information, order information, and individual order details. SQL queries are then used to generate customer order history, product-wise order information, and customer purchase analysis.

1. Create the Required Tables

The database uses four related tables: Customer, Product, Orders, and Order_Details. Primary keys uniquely identify records, while foreign keys establish relationships between the tables.



```
Enter user-name: system
Enter password:
Connected.
SQL> CREATE TABLE Orders (
  2      Order_ID INT PRIMARY KEY,
  3      Customer_ID INT,
  4      Order_Date DATE,
  5      Total_Amount DECIMAL(10,2),
  6      Order_Status VARCHAR(20)
  7  );

Table created.

SQL> DESC ORDERS;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ORDER_ID                                  NOT NULL NUMBER(38)
 CUSTOMER_ID                                        NUMBER(38)
 ORDER_DATE                                         DATE
 TOTAL_AMOUNT                                       NUMBER(10,2)
 ORDER_STATUS                                       VARCHAR2(20)

SQL> CREATE TABLE Order_Details (
  2      Order_Detail_ID INT PRIMARY KEY,
  3      Order_ID INT,
  4      Product_ID INT,
  5      Quantity INT,
  6      Price DECIMAL(10,2),
  7      FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
  8  );

Table created.

SQL> DESC Order_Details
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ORDER_DETAIL_ID                           NOT NULL NUMBER(38)
 ORDER_ID                                           NUMBER(38)
 PRODUCT_ID                                         NUMBER(38)
 QUANTITY                                           NUMBER(38)
 PRICE                                              NUMBER(10,2)

SQL>
```

.

2. Establish Relationships and Constraints

The Orders table is related to Customer through Customer_ID. The Order_Details table is related to Orders through Order_ID. The Order_Details table also stores Product_ID to identify the product included in each order.

- Customer_ID – Primary key in Customer and reference in Orders.
- Order_ID – Primary key in Orders and reference in Order_Details.
- Order_Detail_ID – Primary key in Order_Details.
- Product_ID – Product reference used by Order_Details.

The SQL output below verifies the constraints and the relationship between Orders and Order_Details.

```
  5         od.Quantity,
  6         od.Price
  7  FROM Orders o
  8  JOIN Order_Details od
  9  ON o.Order_ID = od.Order_ID;

  ORDER_ID CUSTOMER_ID ORDER_DETAIL_ID PRODUCT_ID   QUANTITY      PRICE
---------- ----------- --------------- ---------- ---------- ----------
       101           1               1        201          2        500
       101           1               2        202          1        500
       102           2               3        203          1        850
       103           3               4        204          2       1100

SQL> SELECT constraint_name, constraint_type
  2  FROM user_constraints
  3  WHERE table_name IN ('ORDERS', 'ORDER_DETAILS');

CONSTRAINT_NAME                C
------------------------------ -
SYS_C007307                    P
SYS_C007308                    P
SYS_C007309                    R

SQL> SELECT constraint_name, constraint_type
  2  FROM user_constraints
  3  WHERE table_name = 'ORDER_DETAILS'
  4  AND constraint_type = 'R';

CONSTRAINT_NAME                C
------------------------------ -
SYS_C007309                    R

SQL>
```

3. Customer and Order Details

The DESC CUSTOMER command is used to display the structure of the Customer table, including the CUSTOMER_ID and CUSTOMER_NAME columns. A JOIN query is then used to combine the Customer and Orders tables based on the common CUSTOMER_ID. The query successfully displays the customer name, order ID, order date, total amount, and order status. The result shows orders placed by customers such as Ravi, Priya, and Arun, with their respective order amounts and statuses.

```
Headphones


PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
       104
Keyboard


SQL> DESC CUSTOMER;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 CUSTOMER_ID                               NOT NULL NUMBER
 CUSTOMER_NAME                                      VARCHAR2(50)

SQL> SELECT c.Customer_Name,
  2         o.Order_ID,
  3         o.Order_Date,
  4         o.Total_Amount,
  5         o.Order_Status AS Status
  6  FROM Customer c
  7  JOIN Orders o
  8  ON c.Customer_ID = o.Customer_ID;

CUSTOMER_NAME                                      ORDER_ID ORDER_DAT
-------------------------------------------------- ---------- ---------
TOTAL_AMOUNT STATUS
------------ --------------------
Ravi                                                    101 20-SEP-26
        1500 Delivered

Priya                                                   102 21-SEP-26
         850 Shipped

Arun                                                    103 22-SEP-26
        2200 Pending


SQL> SELECT * FROM Order_Details;

ORDER_DETAIL_ID   ORDER_ID PRODUCT_ID   QUANTITY      PRICE
--------------- ---------- ---------- ---------- ----------
```

4. Insert Orders and Order Details

Three orders were inserted with different dates, amounts, and statuses: Delivered, Shipped, and Pending.

```
SQL*Plus: Release 11.2.0.2.0 Production on Fri Sep 25 10:24:23 2026

Copyright (c) 1982, 2014, Oracle.  All rights reserved.

SQL> connect
Enter user-name: system
Enter password:
Connected.
SQL> INSERT INTO Orders
  2  (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  3  VALUES
  4  (101, 1, DATE '2026-09-20', 1500.00, 'Delivered');

1 row created.

SQL>
SQL> INSERT INTO Orders
  2  (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  3  VALUES
  4  (102, 2, DATE '2026-09-21', 850.00, 'Shipped');

1 row created.

SQL>
SQL> INSERT INTO Orders
  2  (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  3  VALUES
  4  (103, 3, DATE '2026-09-22', 2200.00, 'Pending');

1 row created.

SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM Orders;

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ----------- --------- ------------ --------------------
       101           1 20-SEP-26         1500 Delivered
       102           2 21-SEP-26          850 Shipped
       103           3 22-SEP-26         2200 Pending

SQL> INSERT INTO Order_Details
```

The individual products, quantities, and prices for each order were stored in Order_Details.

```
SQL> SELECT * FROM Orders;

  ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
---------- ----------- --------- ------------ --------------------
       101           1 20-SEP-26         1500 Delivered
       102           2 21-SEP-26          850 Shipped
       103           3 22-SEP-26         2200 Pending

SQL> INSERT INTO Order_Details
  2  (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3  VALUES
  4  (1, 101, 201, 2, 500.00);

1 row created.

SQL>
SQL> INSERT INTO Order_Details
  2  (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3  VALUES
  4  (2, 101, 202, 1, 500.00);

1 row created.

SQL>
SQL> INSERT INTO Order_Details
  2  (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3  VALUES
  4  (3, 102, 203, 1, 850.00);

1 row created.

SQL>
SQL> INSERT INTO Order_Details
  2  (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  3  VALUES
  4  (4, 103, 204, 2, 1100.00);

1 row created.

SQL>
SQL> COMMIT;

Commit complete.
```

5. Generate Customer Order History Reports

Report 1: Customer Order History

Purpose: To display the customer name, order ID, order date, total amount, and order status.

```
SQL> SELECT c.Customer_Name,
  2         o.Order_ID,
  3         o.Order_Date,
  4         o.Total_Amount,
  5         o.Order_Status AS Status
  6  FROM Customer c
  7  JOIN Orders o
  8  ON c.Customer_ID = o.Customer_ID;

CUSTOMER_NAME                                      ORDER_ID ORDER_DAT
-------------------------------------------------- ---------- ---------
TOTAL_AMOUNT STATUS
------------ --------------------
Ravi                                                    101 20-SEP-26
        1500 Delivered

Priya                                                   102 21-SEP-26
         850 Shipped

Arun                                                    103 22-SEP-26
        2200 Pending
```

Report 2: Product-wise Order Report

Purpose: To find the number of times each product was ordered and the total quantity sold.

```
  2         COUNT(Order_ID) AS Number_Of_Times_Ordered,
  3         SUM(Quantity) AS Total_Quantity_Sold
  4  FROM Order_Details
  5  GROUP BY Product_ID
  6  ORDER BY Product_ID;

PRODUCT_ID NUMBER_OF_TIMES_ORDERED TOTAL_QUANTITY_SOLD
---------- ---------------------- ------------------
       201                      1                  2
       202                      1                  1
       203                      1                  1
       204                      1                  2

SQL> SELECT c.Customer_Name,
  2         COUNT(o.Order_ID) AS Total_Orders
  3  FROM Customer c
  4  JOIN Orders o
  5  ON c.Customer_ID = o.Customer_ID
  6  GROUP BY c.Customer_ID, c.Customer_Name
  7  ORDER BY Total_Orders DESC;

CUSTOMER_NAME                                      TOTAL_ORDERS
-------------------------------------------------- ------------
Arun                                                          1
Ravi                                                          1
Priya                                                         1

SQL> SELECT c.Customer_Name,
  2         SUM(o.Total_Amount) AS Total_Spending
  3  FROM Customer c
  4  JOIN Orders o
  5  ON c.Customer_ID = o.Customer_ID
  6  GROUP BY c.Customer_ID, c.Customer_Name
  7  ORDER BY Total_Spending DESC;

CUSTOMER_NAME                                      TOTAL_SPENDING
-------------------------------------------------- --------------
Arun                                                         2200
Ravi                                                         1500
Priya                                                         850

SQL> SELECT c.Customer_Name,
  2         AVG(o.Total_Amount) AS Average_Order_Value
  3  FROM Customer c
```

Report 3: Customer Purchase Analysis

This report contains three analyses: customers with maximum orders, total spending by each customer, and average order value.

3.1 Customers with Maximum Orders

The query counts the orders placed by each customer and sorts the result by the number of orders.

```
SQL> SELECT c.Customer_Name,
  2         COUNT(o.Order_ID) AS Total_Orders
  3  FROM Customer c
  4  JOIN Orders o
  5  ON c.Customer_ID = o.Customer_ID
  6  GROUP BY c.Customer_ID, c.Customer_Name
  7  ORDER BY Total_Orders DESC;

CUSTOMER_NAME                                          TOTAL_ORDERS
-------------------------------------------------- ------------
Arun                                                              1
Ravi                                                              1
Priya                                                             1
```

3.2 Total Spending by Each Customer

The query calculates the sum of Total_Amount for each customer.

```
SQL> SELECT c.Customer_Name,
  2         SUM(o.Total_Amount) AS Total_Spending
  3  FROM Customer c
  4  JOIN Orders o
  5  ON c.Customer_ID = o.Customer_ID
  6  GROUP BY c.Customer_ID, c.Customer_Name
  7  ORDER BY Total_Spending DESC;

CUSTOMER_NAME                                     TOTAL_SPENDING
-------------------------------------------------- --------------
Arun                                                        2200
Ravi                                                        1500
Priya                                                        850
```

3.3 Average Order Value

The query calculates the average order value for each customer.

```
SQL> SELECT c.Customer_Name,
  2         AVG(o.Total_Amount) AS Average_Order_Value
  3  FROM Customer c
  4  JOIN Orders o
  5  ON c.Customer_ID = o.Customer_ID
  6  GROUP BY c.Customer_ID, c.Customer_Name
  7  ORDER BY Average_Order_Value DESC;

CUSTOMER_NAME                                     AVERAGE_ORDER_VALUE
-------------------------------------------------- -------------------
Arun                                                             2200
Ravi                                                             1500
Priya                                                             850
```

6. Key Results

- Customer Order History was generated successfully for Ravi, Priya, and Arun.
- The Product-wise Order Report calculated order frequency and total quantity sold for each product.

- The customer order count, total spending, and average order value were calculated using aggregate functions.
- JOIN, GROUP BY, COUNT, SUM, and AVG were used to generate the required reports.

7. Conclusion

The Week 4 SQL project was successfully completed using Oracle SQL. The project demonstrates relational table creation, primary and foreign key relationships, data insertion, JOIN operations, and aggregate functions. The generated reports provide a clear view of customer order history, product-wise sales quantities, and customer purchase behaviour.